

Sistema de gestión de pedidos de comida online

Grupo 8: Pavez, Tuesta, Villalba

Índice

1. Etapa 1: implementación del modelo relacional	2
1.1. Elección del modelo	2
1.2. Implementación en PostgreSQL	3
1.3. Poblado de datos	4
2. Etapa 2: SQL avanzado	4
2.1. Funciones de ventana	4
2.2. Funciones estadísticas	5
2.3. Análisis de performance y optimización	5
3. Etapa 3: Procesamiento distribuido con Apache Spark	6
3.1. MapReduce con Spark RDDs	6
3.2. Consideraciones arquitectónicas: Lazy Evaluation en Spark	9
4. Etapa 4: Persistencia Políglota — NoSQL	9
4.1. Redis	9
4.1.1. Modelo clave-valor y hashes: datos maestros y estado operacional	9
4.1.2. Listas como historial: cinta de restaurantes visitados	10
4.1.3. TTL: datos con tiempo de vida	10
4.2. MongoDB	11
4.2.1. Diseño de colecciones	11
4.2.2. 4.2.2 Operaciones CRUD	11
4.2.3. Pipelines de agregación	12
4.2.4. Reflexión técnica y hallazgos de negocio	12
A. Anexo A: diagrama entidad-relación (DER)	14

1. Etapa 1: implementación del modelo relacional

1.1. Elección del modelo

Para el desarrollo de este trabajo práctico, se seleccionó el dominio de un sistema de gestión de pedidos de comida online (delivery). El ecosistema propuesto simula una plataforma donde convergen tres actores principales: clientes, locales gastronómicos y una flota logística.

Descripción del universo del negocio

El modelo de negocio se estructura de la siguiente manera: un **usuario** registrado en la plataforma puede explorar un catálogo de **platos** ofrecidos por diversos **restaurantes**. Cuando el usuario decide realizar una compra, el sistema genera un **pedido**. Este pedido actúa como el núcleo transaccional del sistema, vinculando al cliente con el restaurante e incluyendo el detalle exacto de las cantidades y precios históricos de los platos adquiridos.

Al momento de la compra, el usuario puede aplicar una **promoción** temporal, la cual impactará en el monto total abonado. Una vez creado, el pedido inicia su ciclo de vida logístico. En lugar de tener un único estado estático, el flujo se registra en un **historial**, permitiendo rastrear temporalmente cuándo el pedido fue creado, cuándo el restaurante comenzó a prepararlo, en qué momento un **repartidor** lo retiró para llevarlo a la **dirección** del cliente, y finalmente, cuándo fue entregado (o si fue cancelado en el proceso).

Decisiones de diseño y alcance

Al traducir esta realidad al diagrama entidad-relación (disponible completo en el Anexo A), se tomaron decisiones sobre los límites del sistema:

- **Inclusiones:** se modeló a detalle el historial logístico y el registro de precios unitarios históricos dentro de cada pedido, lo que asegura que si un restaurante cambia el precio de un plato en el futuro, los pedidos del pasado no se vean alterados.
- **Exclusiones:** para delimitar el alcance al paradigma relacional transaccional, se dejaron fuera entidades como pasarelas de pago (tarjetas de crédito, facturación externa), asumiendo el pago como un proceso consolidado al generar el pedido. Tampoco se modelaron carritos de compras temporales, registrándose únicamente los pedidos efectivamente realizados.

Reglas de negocio y restricciones (constraints)

Para asegurar que los datos guardados tengan sentido lógico y comercial, el modelo fue restringido por múltiples reglas. Algunas de estas restricciones son directas sobre el valor de un campo, mientras que otras dictan la consistencia de todo el sistema.

A. Restricciones estáticas de dominio (implementadas vía esquema):

1. **Cantidades y precios lógicos:** el `precio_actual` de los platos, la `cantidad` de ítems en el detalle y el `total_abonado` del pedido deben ser siempre mayores o iguales a cero.
2. **Límites de descuentos:** el `porcentaje_descuento` de las promociones debe ser un valor comprendido entre 1 y 100.

3. **Stock de promociones:** el atributo `stock_usos` de la tabla `promociones` debe ser mayor o igual a cero.
4. **Coherencia temporal en promociones:** la fecha de fin de una promoción debe ser estrictamente posterior a su fecha de inicio.
5. **Estados válidos:** el atributo `estado` dentro del historial del pedido solo admite los valores: ‘creado’, ‘en_preparacion’, ‘en_camino’, ‘entregado’ o ‘cancelado’.
6. **Categorías válidas:** el atributo `categoria` en la tabla de restaurantes solo admite los valores definidos por el dominio del negocio: ‘pizzería’, ‘hamburguesería’, ‘sushi’, ‘comida sana’, ‘parrilla’ o ‘heladería’. Esto garantiza la consistencia de los datos sin la necesidad de agregar tablas adicionales.

B. Aserciones complejas de validación lógica:

1. **Vigencia de promociones:** la fecha de creación de un pedido que usó una promoción debe estar comprendida dentro del rango de vigencia (fecha de inicio y fin) de la promoción aplicada.
2. **Cronología del historial:** la fecha y hora de cualquier registro en el historial debe ser estrictamente posterior (o igual) a la fecha de creación del pedido asociado.
3. **Consistencia financiera:** el monto total abonado en la cabecera debe coincidir exactamente con la sumatoria del precio unitario por la cantidad de todos sus ítems, restando el porcentaje de la promoción si aplicase.
4. **Unicidad de estados:** para un mismo pedido, no pueden existir dos registros con el mismo estado en su historial.
5. **Cronología de usuarios:** la fecha de creación de un pedido debe ser posterior a la fecha de registro en la plataforma del usuario que lo realizó.
6. **Integridad del catálogo:** todos los platos en el detalle de un pedido deben pertenecer al identificador del restaurante registrado en la cabecera.
7. **Consistencia logística:** todo pedido que posea un registro de estado ‘en_camino’ o ‘entregado’ debe tener un identificador de repartidor asignado (no nulo).
8. **Máquina de estados finitos:** las transiciones de estado deben respetar un flujo secuencial válido: de ‘creado’ solo a ‘en_preparacion’ o ‘cancelado’; de ‘en_preparacion’ solo a ‘en_camino’ o ‘cancelado’; de ‘en_camino’ solo a ‘entregado’ o ‘cancelado’.

1.2. Implementación en PostgreSQL

La creación de la base de datos se plasmó en el script estructurado `1_creacion_tablas.sql`. Debido a que el motor relacional exige que las referencias existan antes de ser invocadas, el script fue ordenado jerárquicamente. Primero se instanciaron las tablas catálogo independientes (`usuario`, `calle`, `promocion`, `repartidor`) y posteriormente las tablas transaccionales que consolidan claves foráneas (`pedido`, `detalle_pedido`, `historial_estado_pedido`).

En este proceso se definieron los tipos de datos exactos (ej. `TIMESTAMP` para auditorías precisas, `DECIMAL` para finanzas) y se materializaron todas las claves primarias (PK) y foráneas (FK).

Asimismo, las reglas listadas en la sección anterior como “restricciones estáticas de dominio” fueron programadas de manera nativa utilizando constraints de tipo `CHECK` a nivel de tabla.

1.3. Poblado de datos

Las aserciones complejas de validación lógica mencionadas anteriormente (como la máquina de estados o la consistencia financiera cruzada entre tablas) superan la capacidad nativa de las restricciones `CHECK` simples durante la creación del esquema. Por lo tanto, para inyectar un volumen masivo de datos coherentes que respetara esta red de reglas lógicas, se optó por programar un generador en Python.

Este script generó un escenario realista de producción que superó el criterio mínimo de la consigna, alcanzando más de 5.000 pedidos completos con variabilidad en fechas, comportamientos de usuarios recurrentes y diversidad categórica.

Para confirmar que la información inyectada por el generador cumple estrictamente con el modelo propuesto, se desarrollaron consultas de agregación en SQL, estas se encuentran en `etapa_2_consultas_avanzadas/validacion_consistencia.sql`

2. Etapa 2: SQL avanzado

2.1. Funciones de ventana

Para profundizar en las capacidades analíticas del motor relacional, se diseñaron consultas que requieren evaluar relaciones entre múltiples registros pertenecientes a una misma entidad lógica, algo que excede las capacidades de un agrupamiento tradicional (`GROUP BY`).

Consulta 1: eficiencia en tiempos de preparación por restaurante

Pregunta de negocio: ¿Cuáles son los cinco restaurantes más eficientes en su cocina, midiendo el tiempo exacto que demoran desde que comienzan a preparar un pedido hasta que este es despachado?

Justificación técnica: calcular la diferencia temporal entre dos estados de un mismo pedido es complejo en SQL puro porque los estados (`'en_preparacion'` y `'en_camino'`) residen en filas distintas. Un enfoque tradicional obligaría a realizar autouniones ineficientes de la tabla consigo misma. En cambio, se utilizó la función de ventana `LEAD()` para alinear temporalmente el estado actual con el siguiente en una sola lectura:

```
LEAD(fecha_hora) OVER (PARTITION BY id_pedido ORDER BY fecha_hora)
```

Posteriormente, se calculó la diferencia y se agrupó el promedio por restaurante.

Al ejecutar la consulta, se observó que el restaurante ‘La esquina de Moreno’ lidera la eficiencia con un promedio de 10.95 minutos de preparación.

Consulta 2: fidelización de clientes por categoría

Pregunta de negocio: ¿Quiénes son los 3 clientes más con mayor volumen de dinero gastado dentro de cada categoría gastronómica (ej. pizzería, sushi, parrilla)?

Justificación técnica: un agrupamiento clásico (`GROUP BY`) permite calcular el total gastado por cada usuario en cada categoría, lo cual se resolvió en una expresión de tabla común (CTE) inicial. Sin embargo, el motor SQL no permite utilizar funciones de ventana directamente en la cláusula `WHERE` para realizar filtros. Por ello, en una subconsulta intermedia se empleó la función `RANK()` particionada por categoría (`PARTITION BY categoria`) y ordenada descendientemente por el nivel de gasto. Esto asigna una posición posicional a cada cliente dentro de su nicho, permitiendo que la consulta externa filtre fácilmente el podio exigiendo que el rango sea menor o igual a 3.

2.2. Funciones estadísticas

Se seleccionó la tabla `pedido` como caso de estudio analítico. La elección se fundamenta en que esta entidad representa el núcleo transaccional del sistema y concentra la diversidad estructural necesaria. A continuación, se detallan los hallazgos:

- **C1 (Complejidad y unicidad):** la evaluación de calidad de datos demostró un 100 % de completitud valores no nulos para la columna `id_usuario` y de un 24.9 % para `id_promocion`. Adicionalmente, el conteo de valores distintos (`COUNT DISTINCT`) permitió dimensionar la variedad de la muestra generada, confirmando que la lógica de inserción del script no produce duplicados anómalos.
- **C2 (Estadística descriptiva y detección de outliers):** al analizar la columna financiera `total_abonado` utilizando percentiles continuos (`PERCENTILE_CONT`), se detectó una mediana de \$34000 y un promedio de \$37826. El uso encadenado de expresiones `WITH` (CTE) permitió calcular dinámicamente los límites del rango intercuartílico, logrando aislar matemáticamente 25 pedidos clasificados como outliers. Asimismo, se validó la integridad de la facturación al confirmar un 0 % de valores negativos o en cero.
- **C3 (Frecuencia categórica y distribución de mercado):** el análisis de la distribución de pedidos por restaurante reveló que ningún local individual posee un monopolio transaccional; de hecho, el líder (restaurante 48) concentra apenas el 2.40 % de las ventas. El top 10 de locales más solicitados suma menos de una cuarta parte de las ventas, mientras que la categoría colapsada “Resto” agrupa el 76.72 % del volumen total. Esto indica que los usuarios de la plataforma no se limitan a un puñado de cadenas populares, sino que consumen de una base extremadamente amplia y variada de opciones gastronómicas.

2.3. Análisis de performance y optimización

A medida que el volumen de datos transaccionales crece, el costo de las operaciones analíticas se incrementa. Para auditar el rendimiento del motor relacional, se analizó el plan de ejecución de la consulta que calcula la eficiencia en tiempos de preparación (sección 2.1), utilizando la sentencia `EXPLAIN ANALYZE`.

Diagnóstico inicial (sin optimizar): El planificador de PostgreSQL arrojó un tiempo de ejecución de 25.77 ms y un costo acumulado tope de 2508. Al desglosar el árbol de

ejecución, se detectó que el cuello de botella se encuentra en la preparación de los datos para la función de ventana `LEAD()`. El motor se ve obligado a realizar un *Sequential Scan* (lectura secuencial completa) sobre las 18.917 filas de la tabla `historial_estado_pedido`, seguido de una operación de ordenamiento en memoria extremadamente costosa (*quicksort* de 2060kB) para cumplir con la directiva `PARTITION BY id_pedido ORDER BY fecha_hora`.

```
-> WindowAgg (cost=1667.98..2093.61 rows=18917 ...)  
-> Sort (cost=1667.98..1715.27 rows=18917 ...)  
    Sort Key: h.id_pedido, h.fecha_hora  
-> Seq Scan on historial_estado_pedido h (cost  
    =0.00..324.17 ...)
```

Solución propuesta (indexación compuesta): Para evitar que PostgreSQL tenga que leer todo de forma desordenada y luego ordenarlo en la memoria RAM en cada ejecución, se implementó un índice compuesto que refleja exactamente la estructura de la función de ventana:

```
CREATE INDEX idx_historial_ventana ON historial_estado_pedido (  
    id_pedido, fecha_hora);
```

Análisis comparativo: Tras la creación de esta estructura, se volvió a ejecutar el planificador de consultas, evidenciando un cambio radical en la estrategia de acceso a los datos:

```
-> WindowAgg (cost=0.29..1276.26 rows=18917 ...)  
-> Index Scan using idx_historial_ventana on  
    historial_estado_pedido h (cost=0.29..897.92 ...)
```

Como se observa en el nuevo plan, el motor descartó el escaneo secuencial masivo y el costoso nodo de ordenamiento (*Sort* en memoria) desapareció por completo de la subconsulta interior. En su lugar, PostgreSQL utilizó un *Index Scan*. Al leer este índice, el motor obtuvo las 18.917 filas ya agrupadas por pedido y ordenadas cronológicamente de forma nativa, inyectándolas directamente a la función de ventana.

El costo general de la consulta se redujo de 2508.47 a 1668.00 (una mejora algorítmica aproximada del 33%), y el tiempo de ejecución descendió de 25.77 ms a 23.82 ms. Aunque la diferencia en tiempo absoluto es marginal debido al tamaño controlado de este volumen de prueba, la drástica reducción del costo de procesamiento demuestra que indexar las columnas particionadas en las funciones de ventana es vital para garantizar la escalabilidad analítica del sistema.

3. Etapa 3: Procesamiento distribuido con Apache Spark

3.1. MapReduce con Spark RDDs

Para procesar volúmenes de información que exceden la capacidad vertical de un motor relacional convencional (como el análisis matricial o de texto libre), se implementó un entorno de procesamiento distribuido utilizando la API de RDDs de PySpark. Los datos transaccionales de la Etapa 1 fueron exportados a formato CSV para ser consumidos por el pipeline de Spark.

A continuación, se detallan las tres consultas analíticas implementadas bajo el paradigma MapReduce.

Consulta 1: análisis léxico de tendencias de consumo

Pregunta de negocio: ¿Cuáles son los ingredientes o conceptos comerciales más recurrentes en las descripciones de los platos ofrecidos? Esta métrica es fundamental para identificar macro-tendencias de consumo, estrategias de venta de los locales y orientar el alta de nuevos restaurantes.

Justificación: analizar texto libre (tokenización y filtrado de palabras vacías o *stop-words*) es una operación prohibitiva en SQL, pero naturalmente paralela en un ecosistema distribuido.

- **Fase de Map:** el RDD inicial lee el catálogo de platos. Se aplica una transformación `map` para normalizar el texto (minúsculas, eliminación de signos de puntuación) seguido de un `flatMap` para dividir las descripciones en palabras individuales, excluyendo los conectores gramaticales. Se emite la estructura `(palabra, 1)`.
- **Fase de Reduce:** se ejecuta la transformación `reduceByKey(lambda a, b: a + b)`, la cual consolida todos los nodos agrupando por la clave léxica (`palabra`) y sumando sus ocurrencias.

Resultado obtenido: El procesamiento distribuido arrojó el siguiente top 10 de términos más frecuentes en todo el catálogo:

- elección: 24 menciones
- tomate: 23 menciones
- corte: 20 menciones
- doble: 18 menciones
- piezas: 16 menciones
- palta: 15 menciones
- masa: 12 menciones
- extra: 12 menciones
- frita: 12 menciones
- cortada: 12 menciones

El análisis léxico revela tendencias claras en el comportamiento del mercado gastronómico simulado. Por un lado, domina la personalización (con términos como “elección”, lo que indica que los locales ofrecen platos armables a medida. Por otro lado, resalta una fuerte estrategia de upselling o venta incremental (“doble”, “extra”). En cuanto a los insumos orgánicos, el tomate y la palta son los ingredientes más repetidos, seguidos por métodos de preparación como “frita” y bases como “masa”.

Consulta 2: filtro colaborativo de clientela compartida

Pregunta de negocio: ¿Qué pares de restaurantes suelen ser consumidos por los mismos usuarios? Esta métrica permite inferir afinidad gastronómica oculta para motores de recomendación, independientemente de la categoría formal del local.

Justificación: este problema requiere generar combinaciones cruzadas por cada usuario. En SQL, requeriría un *JOIN* múltiple altísimamente ineficiente a medida que la base de clientes escala.

- **Fase de Map (preparación):** se extraen los pedidos emitiendo la tupla (*id_usuario*, *id_restaurante*) y se aplica un *distinct()* para contar solo si el usuario probó el restaurante, sin importar cuántas veces.
- **Fase de Reduce y mapeo cruzado:** se aplica un *groupByKey()* para agrupar en una lista todos los restaurantes que visitó un usuario determinado. Luego, se ejecuta un *flatMap* que calcula todas las combinaciones posibles de pares dentro de esa lista (ej. (*restaurante_A*, *restaurante_B*)). Finalmente, un segundo *reduceByKey* suma las coincidencias totales de cada par de restaurantes en el ecosistema global.

Resultado obtenido: El procesamiento reveló las afinidades latentes más fuertes del sistema, destacando el siguiente Top 5 de pares de restaurantes con mayor clientela compartida:

- Restaurantes 3 y 18: 22 usuarios en común.
- Restaurantes 10 y 21: 21 usuarios en común.
- Restaurantes 14 y 30: 20 usuarios en común.
- Restaurantes 25 y 38: 20 usuarios en común.
- Restaurantes 5 y 27: 19 usuarios en común.

La obtención de esta matriz de co-ocurrencia valida la viabilidad de implementar un sistema de recomendación basado en filtrado colaborativo. Al confirmar estadísticamente que los restaurantes 3 y 18 comparten una base sólida de clientes, la plataforma puede ejecutar acciones predictivas: si un usuario nuevo realiza un pedido en el local 3, el motor le sugerirá activamente el local 18 en su próxima sesión, aumentando la probabilidad de conversión. Asimismo, habilita el diseño de campañas de marketing cruzado entre locales que no necesariamente pertenecen a la misma categoría formal.

Consulta 3: segmentación de “cazadores de promociones”

Pregunta de negocio: ¿Qué usuarios recurrentes (mínimo 3 pedidos) presentan la mayor tasa de dependencia hacia los cupones de descuento respecto a su volumen histórico de compras?

Justificación: este cálculo requiere manejar contadores paralelos a nivel de usuario consolidado para luego calcular proporciones y aplicar filtros. Es una operación analítica donde el paradigma *map-reduce* destaca por su capacidad de agregación distribuida.

- **Fase map:** se evalúa cada transacción. Si el pedido tiene un identificador de promoción, se emite (*id_usuario*, (1, 1)) donde el formato es: (*pedido_total*, *pedido_con_promo*). Si no tiene promoción, se emite (*id_usuario*, (1, 0)).

- **Fase reduce:** un `reduceByKey` acumula matemáticamente las tuplas sumando ambos contadores de forma paralela. Posteriormente, una transformación `mapValues` calcula el porcentaje exacto de uso de descuentos, y un `filter` descarta a los usuarios esporádicos (menos de 3 pedidos).

Resultado obtenido: La ejecución permitió identificar a los usuarios con mayor sensibilidad al precio, revelando el siguiente top 5 de clientes con mayor dependencia a las ofertas:

- Usuario 438: 100.0 % de sus pedidos con cupón.
- Usuario 144: 100.0 % de sus pedidos con cupón.
- Usuario 675: 100.0 % de sus pedidos con cupón.
- Usuario 97: 85.71 % de sus pedidos con cupón.
- Usuario 645: 83.33 % de sus pedidos con cupón.

Los resultados evidencian un segmento de clientes hiper-sensibles al precio que prácticamente no interactúan con la plataforma si no existe un incentivo económico. Desde una perspectiva de rentabilidad, estos “cazadores de ofertas” representan un desafío operativo: generan volumen de transacciones, pero con márgenes de ganancia potencialmente nulos o negativos. Identificarlos a nivel de base de datos permite al equipo de marketing excluirlos de futuras promociones masivas, optimizando la inversión publicitaria.

3.2. Consideraciones arquitectónicas: Lazy Evaluation en Spark

Durante el desarrollo de este pipeline, se comprobó la naturaleza perezosa (*lazy evaluation*) del motor de Spark.

A diferencia de un motor relacional interactivo, Spark no procesa los datos físicamente cuando se declaran las transformaciones (como `map`, `flatMap`, `filter` o incluso `reduceByKey`). Estas instrucciones únicamente construyen un Grafo Acíclico Dirigido (DAG) en memoria, trazando el plan lógico de ejecución.

El cómputo real y el consumo de recursos en el clúster (lectura de disco, movimiento de datos por la red e instanciación de memoria) se desencadenan exclusivamente cuando se invoca una acción. En los casos expuestos en este informe, el motor solo ejecutó el DAG cuando se llamaron comandos finales como `collect()`, `take(10)` o `saveAsTextFile()`, optimizando el recorrido de los datos al consolidar transformaciones adyacentes en una sola pasada de ejecución.

4. Etapa 4: Persistencia Políglota — NoSQL

4.1. Redis

4.1.1. Modelo clave-valor y hashes: datos maestros y estado operacional

Escenario de negocio: la plataforma necesita acceder de forma permanente y a muy alta frecuencia a ciertos datos de consulta inmediata: la información de perfil de cada usuario, el estado actual de cada pedido en su ciclo de vida y la disponibilidad de

cada repartidor. Estos datos se leen en muchas interacciones del sistema por lo que es conveniente que su consulta sea eficiente.

Justificación técnica: se modelaron los datos de usuario como estructuras tipo *Hash* (`usuario:ID`), ya que agrupan múltiples campos (nombre, email, teléfono, fecha de registro) bajo una sola clave, permitiendo recuperar el perfil completo con una única operación `HGETALL` o modificar un atributo puntual con `HSET` sin reescribir el resto. En cambio, el estado del pedido (`idpedido:ID`) y el estado del repartidor (`id_repartidor:ID`) se modelaron como pares clave-valor simples, dado que representan un único string volátil que cambia constantemente (por ejemplo, de `en_preparacion` a `en_camino`, o de `activo` a `inactivo`). Mantener estos indicadores en memoria evita consultar el disco del motor relacional cada vez que el sistema necesita evaluar y actualizar la disponibilidad de la flota o el avance de un envío.

4.1.2. Listas como historial: cinta de restaurantes visitados

Escenario de negocio: Un pequeño historial de los últimos restaurantes visitados siempre existe en las plataformas modernas de delivery por ser de utilidad al usuario. Este historial de navegación debe actualizarse en tiempo real con cada interacción del cliente en la aplicación, en nuestro caso conservamos únicamente los últimos diez restaurantes visitados.

Justificación técnica: implementar esta funcionalidad en una base de datos relacional implicaría ejecutar una sentencia `INSERT` seguida de un `DELETE` (para mantener el límite estricto de 10 elementos) por cada clic de navegación de cada usuario. Este volumen de operaciones de escritura y borrado haría poco eficiente el mantenimiento del historial. Al utilizar una lista de Redis, la estructura reside completamente en la memoria principal (RAM), permitiendo escrituras de latencia ultrabaja mediante `LPUSH` y acotando el historial a sus elementos más recientes con un costo algorítmico mínimo. La estructura de lista resulta natural aquí porque preserva el orden cronológico de las visitas y permite tanto recuperar la secuencia completa (`LRange`) como eliminar elementos puntuales que el usuario decida descartar (`LRem`).

4.1.3. TTL: datos con tiempo de vida

Escenario de negocio: numerosos datos del dominio solo tienen sentido durante una ventana temporal acotada y deben dejar de existir automáticamente una vez vencido su plazo: promociones vigentes por tiempo limitado, asignaciones de pedidos que un repartidor debe aceptar antes de que se reasignen, y el tiempo esperado de preparación de un pedido hasta su despacho.

Justificación técnica: la funcionalidad de expiración nativa de Redis (TTL) permite delegar la gestión del tiempo al propio motor, eliminando la necesidad de procesos externos que vigilen los plazos y purguen registros vencidos. Se implementaron los tres casos representativos:

- **Vigencia de promociones:** las ofertas publicadas por los restaurantes se almacenan con un TTL (por ejemplo, 86.400 segundos para una promoción de un día). Al vencer el plazo, la clave se elimina sola y la promoción deja de aplicarse, sin intervención manual. Una función de verificación consulta el tiempo restante mediante el comando `TTL` y reporta si la promoción sigue vigente o ya expiró.

- **Tiempo para aceptar un pedido:** cuando el sistema ofrece un pedido para un repartidor, la oferta se registra con un TTL acotado. La presencia de la clave significa que la oferta sigue vigente; su ausencia indica que el repartidor no respondió a tiempo y que el pedido debe ofrecerse a otro repartidor.
- **Tiempo límite de preparación:** al crearse un pedido se establece una clave con un margen de 25 minutos. La demora se detecta cuando el sistema consulta el pedido y observa que la clave de tiempo ya expiró (su TTL vale -2) mientras el estado sigue en `en_preparacion`. En ese caso, la función emite una alerta de pedido demorado, caso contrario devuelve que el pedido está a tiempo y muestra los minutos restantes.

4.2. MongoDB

4.2.1. Diseño de colecciones

Para demostrar las capacidades del modelo documental y contrastarlo con el esquema relacional de la primera etapa, se diseñaron dos colecciones principales priorizando la desnormalización y la eficiencia de lectura.

Colección de restaurantes (catálogo)

- **Datos embebidos:** el catálogo de platos (menú) se embebió directamente como un arreglo de sub-documentos, eliminando los *JOINS* de la etapa relacional.
- **Enriquecimiento y evolución:** el atributo relacional único de categoría se transformó en un arreglo dinámico (`categorias`), permitiendo multiplicidad de etiquetas. Además, se añadió una métrica de `calificacion_promedio` y un sub-documento opcional de `redes_sociales`, demostrando la flexibilidad ante esquemas variables sin generar valores nulos.

Colección de pedidos (transacciones)

- **Datos embebidos y referenciados:** el detalle de los platos consumidos, la dirección de entrega y el historial logístico de estados se embebieron en el pedido, logrando un modelo pre-ensamblado donde toda la transacción vive encapsulada en un solo documento JSON. Por el contrario, `id_restaurante` e `id_usuario` se mantuvieron como referencias para evitar anomalías de actualización masiva si cambian los datos maestros.
- **Campos opcionales:** atributos como `promocion_aplicada` o `id_repartidor` (inexistente si el pedido se canceló al instante) simplemente se omiten en el documento, evitando la obligatoriedad de los valores NULL de PostgreSQL.

4.2.2. Operaciones CRUD

Para validar el esquema de ambas colecciones, se ejecutaron las operaciones de gestión mediante la librería PyMongo:

- **Inserción:** se utilizaron `insert_one` e `insert_many` para ingestar restaurantes con estructuras anidadas y lotes de pedidos cancelados atómicamente.

- **Consultas y proyecciones:** mediante operadores lógicos (`$and`, `$or`) y de comparación (`$gte`, `$lte`), se aislaron tickets de alto valor y locales de alta calificación. Las proyecciones excluyeron intencionalmente los arreglos extensos (menús e historiales) para optimizar el ancho de banda, devolviendo solo métricas clave.
- **Actualización:** se explotó la naturaleza documental utilizando `$push` para inyectar nuevos estados transaccionales y etiquetas sin borrar los anteriores. Se empleó `$set` para ajustar calificaciones y `$inc` para modificar métricas de seguidores y cargos logísticos.
- **Eliminación:** se ejecutaron purgas condicionales (`delete_many` con `$lte`) para limpiar la base de datos de micro-pedidos cancelados y locales con bajo rendimiento.

4.2.3. Pipelines de agregación

Se desarrollaron cuatro pipelines para extraer inteligencia de negocio integrando ambas colecciones.

1. **Competitividad y precios por rubro (catálogo):** utiliza `$unwind` para desarmar categorías y menús, seguido de un `$group` con acumuladores (`$avg`, `$max`) para identificar las ofertas gastronómicas más accesibles.
2. **Auditoría de facturación comercial (cruce relacional):** partiendo de los pedidos, agrupa la recaudación histórica y utiliza `$lookup` para cruzar con el catálogo de restaurantes, proyectando el *top 5* de locales líderes.
3. **Eficiencia de tiempos de entrega:** aísla pedidos entregados (`$match`), calcula la demora logística exacta extrayendo y restando fechas del historial embebido (`$project`), y agrupa por franja horaria para detectar picos operativos.
4. **Rentabilidad de campañas promocionales:** consolida el volumen de uso de cupones por cliente y filtra a los usuarios recurrentes, construyendo un ranking de dependencia a los descuentos.

4.2.4. Reflexión técnica y hallazgos de negocio

Evaluación de arquitecturas de datos

El análisis de las consultas ejecutadas permite contrastar las herramientas tecnológicas del proyecto:

- **Cuándo MongoDB es superior:** resulta extremadamente natural para calcular precios por rubro. Al tener el menú y las categorías embebidas, el motor lee un solo documento. En PostgreSQL, esto obligaría a cruzar tres tablas mediante operaciones *JOIN*, penalizando el rendimiento en disco.
- **Cuándo PostgreSQL es la herramienta ideal:** MongoDB pierde eficiencia en la auditoría de facturación, ya que requiere un `$lookup` intensivo para emular relaciones entre transacciones y catálogos. El núcleo transaccional de PostgreSQL está matemáticamente optimizado para relacionar estas entidades de forma íntegra.

- **El rol de SparkSQL:** resolver las métricas logísticas diarias en Spark implicaría una latencia de orquestación inaceptable frente a la respuesta instantánea de MongoDB. SparkSQL debe reservarse exclusivamente para el procesamiento distribuido de terabytes de datos históricos acumulados.

Al contrastar los pipelines documentales con las métricas globales de la base relacional, se detectó un patrón crítico: el segmento de usuarios con alta dependencia a las promociones genera tickets promedio que oscilan entre \$25.000 y \$31.500, contrastando drásticamente con los \$85.000 de los usuarios premium. Esto confirma que los clientes sensibles al precio utilizan los descuentos para abaratar transacciones de bajo margen, sugiriendo estratégicamente que la empresa debe redirigir su presupuesto de retención hacia la fidelización del segmento de alto valor, en lugar de subsidiar operaciones de baja rentabilidad.

A. Anexo A: diagrama entidad-relación (DER)

